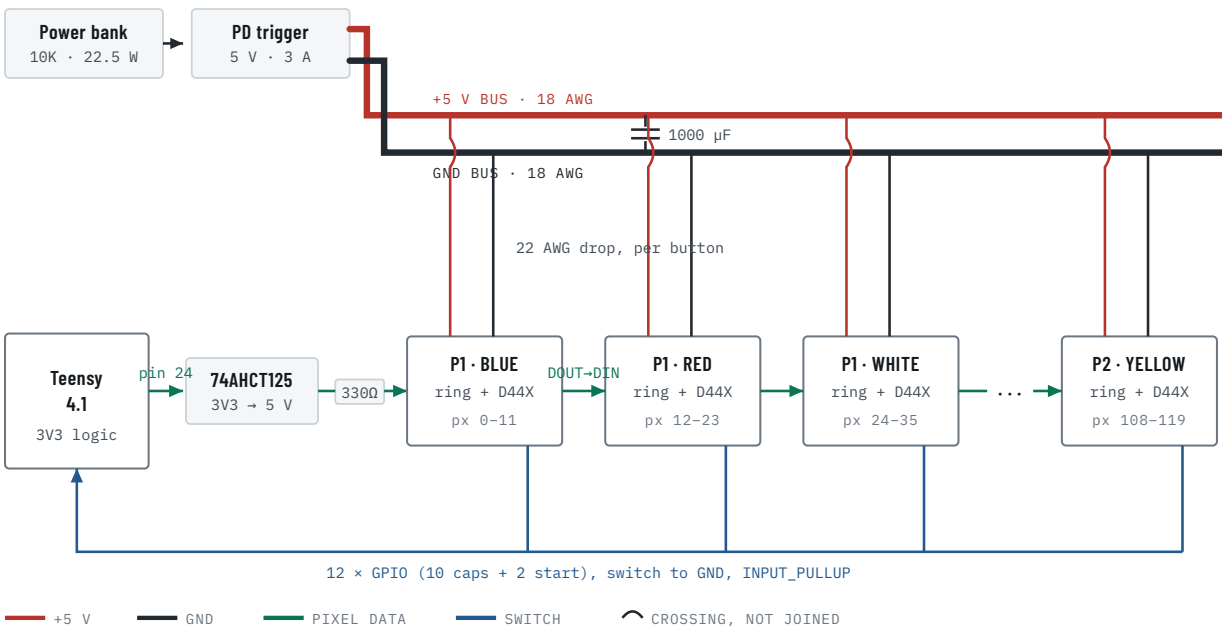
Mole Table Wiring

Ten 60 mm dome buttons carrying a 12-pixel NeoPixel ring and a Cherry D44X switch, plus two unlit start buttons, back to one Teensy 4.1. Pin assignments are mirrored from `firmware/WhackAMole/Config.h`.

> ### The one rule
>
> Power does not daisy-chain through the buttons. Only the data line does. A 12-pixel ring pulls **720 mA** at full white, so chaining 5 V through the button connectors would put **7.2 A** through the first one. Run a separate 5 V/GND bus and tap each button off it.

System topology

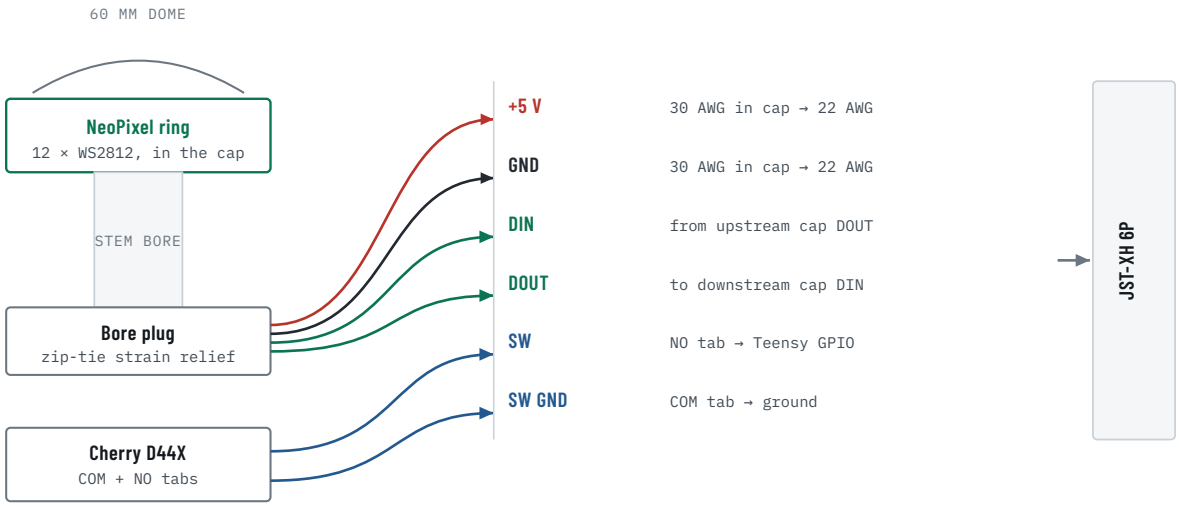
10 lit caps · 120 pixels



Two independent networks. Power fans out from a shared bus so no connector carries more than one ring's current; data runs series through every ring in turn, which is why one unplugged cap goes dark for everything downstream of it. The two start buttons sit outside both networks — switch only.

18 AWG bus, 22 AWG drops 26 AWG point to point 22 AWG returns

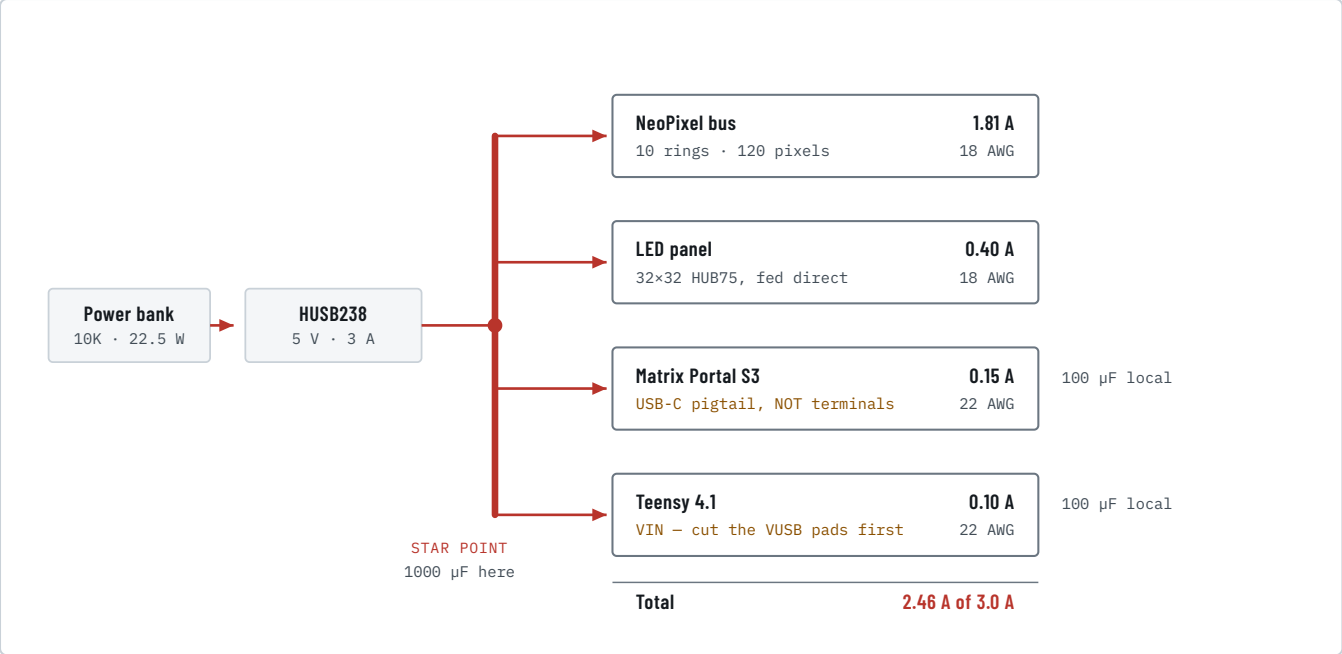
Inside one cap 6 conductors out



Keep the two grounds separate all the way back. A ring at full white returns 720 mA; sharing that return with the switch puts an IR offset across the input the Teensy is trying to read.

Power distribution one supply, four loads

All four loads run from the single 5 V contract. What matters is that they are fed as a **star from one point**, not chained one board to the next. The rings switch current fast; share a thin feed with them and those spikes sag the rail far enough to reset a microcontroller, which presents as random crashes rather than as a power problem.



Four pairs from one point, not a chain. The two annotated loads each need a board modification before they can take bus power safely.

Both microcontrollers need work before they take bus power

Teensy 4.1 — cut the VUSB/VIN pads. They are bridged from the factory, so feeding `VIN` from the bus while USB is plugged into a computer back-feeds the bank into the computer's USB port. This is not optional here: debug mode reports over USB serial, so both will routinely be connected. Cut the two rectangular pads on the underside and confirm with a meter that they are open. It is reversible with solder — and note that afterwards USB no longer powers the board, so it needs the bus even to program.

Matrix Portal S3 — its screw terminals are outputs, not inputs. They are fed from its USB-C port and exist to pass power out to the panel. Adafruit discourages back-feeding them because plugging USB in at the same time can damage the board. Power it through its USB-C instead, from a pigtail carrying 5 V and ground off the bus; the board has its own CC pulldowns, so a dumb source is fine. Unplug that pigtail when connecting a computer. Feed the panel straight from the bus, not through the Portal.

Adafruit publishes no continuous current rating for the HUSB238 breakout — only that the chip negotiates up to 5 A. Nearly 2.5 A through a small board and a 2-pin terminal block is real current, so check it for warmth on the `LED ALL` screen. If it runs hot, take VBUS through a separate high-current path and leave the HUSB238 doing nothing but the negotiation.

Current budget

worst case is a debug screen

DRAW AT 5 V	PIXELS LIT	CURRENT	NOTES
Gameplay, two moles up	24	≈ 0.30 A	Amber, not white. The normal case.
Attract mode	0–24	≈ 0.15 A	Quiescent draw is ~1 mA per pixel.
Debug `LED ALL`, white	120	1.81 A	At `kPixelBrightness = 64`. The most the rings ever pull.
Same, brightness 255	120	7.20 A	Why the ceiling exists.
32×32 LED panel	—	≈ 0.40 A	Scrolling text on black. Every pixel white would be ~2 A, which the game never displays — do not budget against that figure.
Matrix Portal S3 board	—	≈ 0.15 A	The controller itself, separate from the panel it drives.
Teensy + level shifter	—	≈ 0.10 A	
Worst case — `LED ALL` plus the rest	**120**	**≈ 2.46 A**	**12.3 W, against a 15 W supply.**

And what the supply delivers:

SUPPLY	AT 5 V	NOTES
Power bank, 5 V PD contract (`in use`)	3.0 A · 15 W	Straight onto the bus. No conversion, no converter. ~18% headroom over worst case.
Power bank, 9 V PD → XL4015	≈ 3.5 A · 20 W	The path not taken. More headroom, more parts — see the decision record below.
2× 21700 in series → XL4015	sags	Falls below the XL4015's 8 V input floor for most of the discharge.

Two things to verify, not assume

The 3 A contract has to be real. A USB-C source only offers 3 A if the sink asks properly over the CC lines. A plain breakout without the 5.1 kΩ pulldowns gets no VBUS at all on a C-to-C cable; a generic one may only be offered 500 mA. Use a trigger board configured for 5 V, and a **C-to-C cable** — a USB-A-to-C cable has no CC lines and cannot carry a PD contract.

The bank has to stay awake. Power banks shut down below roughly 50–200 mA, and attract mode is the quietest the table ever gets. An active PD contract usually prevents it, but it varies by model — leave the table idling for half an hour before an event, not during one.

Then put a meter on the 5 V rail on the `LED ALL` screen, which is the only state that reaches worst case.

Why there is no converter `decision record`

The table originally ran two 21700 cells in series into an XL4015 buck converter. That pack falls from 8.4 V to 6.0 V across a discharge and the XL4015 needs 8 V at its input, so for most of a session the converter was running below its own input floor. Fixing that is what started this change; it is worth recording why the fix deleted the converter instead of replacing the pack.

OPTION	AT 5 V	ASSESSMENT
3S 21700 pack → XL4015	≈ 4.4 A	Fixes the input floor and nothing else. Still loose cells, still a balance charger, still a pack that can be shorted under a table kids are hitting.
Power bank → 9 V PD → XL4015	≈ 3.5 A	Works, and leaves the most room. Costs a trigger, a converter, a Schottky, and ~1.5 W as heat.
Power bank → 5 V PD (chosen)	3.0 A	Fewest parts. The ceiling is lower and fixed.

What decided it

The current budget, once it was corrected. The first version counted the matrix panel at its all-pixels-white figure of 2.0 A and stacked that against the `LED ALL` debug screen — two worst cases that never occur together. The panel shows scrolling text on black, nearer 0.4 A. Real worst case is 2.46 A across all four loads, and gameplay is nearer 1 A, so the converter was converting power the table did not need converted.

What it buys

- Three fewer components in the power path: the converter, its Schottky, and the wiring between them.

- No conversion loss. The XL4015 is non-synchronous and runs 87–93%, so roughly 1.5 W would have gone to heat at worst case.
- The 8 V input floor stops being a constraint that can ever bite, rather than one worked around.
- A regulated 5 V rail from the bank's own converter, which is a better one than the XL4015.

What it costs

- **A hard 15 W ceiling with no room to grow.** The 9 V path would have given 20 W.
- **Audio is still to come.** A MAX98357A pulls around 0.6 A on peaks. Gameplay plus audio is ~1.6 A and fine, but `LED ALL` at full brightness with sound playing would exceed 3 A. That collision only exists in a diagnostic state, which is why it was accepted.
- **No regulation against cable drop.** Whatever the USB cable and connector lose is lost from the rail, rather than being corrected by a converter downstream of it.

How to reverse it

Change the trigger's configuration resistor from 5 V to 9 V and put the XL4015 back between it and the bus. One resistor and one board — which is the reason this was a comfortable decision to make early, before any of it is soldered down.

Chain order `must match Config.h`

The physical daisy-chain order and the pixel indices in `Config.h` are the same fact written twice. Wire them in this sequence or the wrong cap lights.

POSITION	BUTTON	PIXELS	SWITCH PIN
1–5	P1 mole caps — blue, red, white, green, yellow	0–59	2, 3, 4, 5, 9
6–10	P2 mole caps — blue, red, white, green, yellow	60–119	11, 12, 14, 15, 16
—	P1 start · P2 start (`no lamp`)	none	22 · 25

The start buttons are switch-only

They are plain arcade buttons with no lamp, so they contribute a switch and nothing else: two conductors, no ring, and no place in the pixel chain. `kPixelCount` is `kButtonsPerPlayer × 2 × kPixelsPerButton` — 120, not 144. Allocating pixels for them would shift nothing electrically but would leave the chain expecting two rings that are not there.

Teensy 4.1 pins `from Config.h`

FUNCTION	PINS	NOTES
Display link, Serial1	`0 RX · 1 TX`	To the Matrix Portal at 115200.
Pixel data	`24`	Serial6 TX. WS2812Serial only drives certain pins.
P1 buttons	`2, 3, 4, 5, 9`	All `INPUT_PULLUP`, switch to ground.
P2 buttons	`11, 12, 14, 15, 16`	
Start buttons	`22 · 25`	
Reserved for audio	`6, 7, 10, 18, 19, 20, 21, 23`	Do not use. Pin 7 is I2S data, which is why the display moved off Serial2.
PRNG seed	`17`	Left floating deliberately.

Parts `beyond the buttons`

Part numbers are what was ordered, not the only thing that works — but the level shifter is worth matching exactly, for the reason in the note below the table.

PART	QTY	ORDER AS	WHY
USB-C PD sink	1	`Adafruit 5807`	HUSB238, configured for **5 V** and **3 A** by solder jumper — not I2C, which cannot be set before the Teensy has power. The current jumper ships at 1 A; leave it there and the source may cut power when the table draws 2.5 A.
Level shifter, 74**AHCT**125	2	`TI SN74AHCT125N` `DK 296-4655-5-ND`	Teensy is 3.3 V; WS2812B wants 0.7×VDD on data. 14-DIP, through-hole. See the note below.
14-pin DIP socket	2	—	On hand-soldered stripboard, turns a desoldering job into a pull.
330 Ω resistor, ¼ W	10	`any film, 5%`	Series on the data line at the first pixel.
1000 µF electrolytic, **16 V**	1–2	`radial, low-ESR`	Across 5 V and ground at the star point. 16 V, not 6.3 V — convention is roughly twice the rail, so a transient has somewhere to go.
100 µF electrolytic, 16 V	3	`radial`	Local bulk at the Teensy and the Matrix Portal.
0.1 µF ceramic, 50 V	10	—	One across pins 14 and 7 of the level shifter, as close to the chip as it will sit.
0.187" female spade crimps	30	`insulated, 22–18 AWG`	Onto the D44X tabs. 24 needed; the spares are for the ones you crimp badly.
JST-XH 6-pin pairs	10	`housing + header + contacts`	One per lit cap, so it unplugs for service.
JST-XH 2-pin pairs	2	`as above`	The start buttons — switch only.
Lever nuts	6–8	`WAGO 221-415`	The star point and the 5 V/GND bus taps. Easier than a barrier strip and reusable.
USB-C pigtail, 5 V + GND	1	—	Powers the Matrix Portal through its USB-C port, since its screw terminals are outputs.
USB-C to USB-C cable	1	—	An A-to-C cable has no CC lines and cannot carry a PD contract at all.

PART	QTY	ORDER AS	WHY
18 AWG stranded, red + black	`10 ft ea`	—	The 5 V/GND bus and the panel feed.
22 AWG stranded, red + black	`25 ft ea`	—	Per-cap power drops, switch returns, microcontroller feeds.
30 AWG silicone, 4 colours	`10 ft ea`	—	Inside the lit caps only; it has to survive the 3 mm of plunger travel.

> ### The letter that matters is the T
>
> Order `AHCT`, never `AHC` or `HC`. The **T** means TTL input thresholds: V_{IH} is 2.0 V, so a 3.3 V signal reads reliably as HIGH while the chip runs from 5 V and drives a 5 V output. Without it, V_{IH} becomes $0.7 \times V_{CC}$ = 3.5 V and the Teensy's 3.3 V output sits *below* the threshold — which fails intermittently and with temperature rather than outright, and is miserable to chase.

Wiring the level shifter

Four gates, one used. The asymmetry to check twice: pin 1 goes to **ground** because it enables the gate you want, while pins 4, 10 and 13 go to $\mathbf{V_{CC}}$ because `OE` is active-low and HIGH disables. Unused inputs and unused output-enables pull opposite directions.

PIN	NAME	CONNECT TO	WHY
1	1OE	**GND**	Active-low; grounding it enables the one gate in use.
2	1A	Teensy pin 24	3.3 V pixel data in.
3	1Y	330 Ω → first pixel DIN	5 V pixel data out.
4	2OE	**V_{CC}**	Gate 2 unused: output disabled, input held.
5	2A	GND	
6	2Y	—	Genuinely open. That is what tri-state disabled means.
7	GND	Ground	The chip's own reference. See the note below.
8	3Y	—	Open.
9	3A	GND	Gate 3 unused.
10	3OE	**V_{CC}**	
11	4Y	—	Open.
12	4A	GND	Gate 4 unused.
13	4OE	**V_{CC}**	
14	V_{CC}	+5 V	0.1 µF from here to pin 7, as close to the chip as it will sit.

Tie the unused inputs locally, not to the star point

Ground them to **pin 7 of this chip**, over the shortest path on the board — not with a wire running back to the bus. During pixel switching the ground bus carries amps of fast-changing current, and wire has inductance, so ground at the star point and ground at the chip differ by a couple hundred millivolts transiently. An input referenced to a distant ground can still bounce relative to its own chip's ground. Short and local is the point.

Tying the unused inputs to V_{CC} instead of GND works identically; convention is GND. What matters is only that they are not left floating — CMOS inputs that float oscillate, draw current, and inject noise into the gate you are actually using.

If the pixel chain is ever split into two, one per player, gate 2 is already sitting there: wiring the socket so enabling it means moving two wires costs nothing now.

Build order `test as you go`

01 **Do the two board modifications first**
Cut the Teensy's VUSB/VIN pads and confirm with a meter. Make the Matrix Portal's USB-C power pigtail. Both are easier on the bench than under a table, and the Teensy cut changes how you program it.

02 **Wire one cap completely**
Ring, switch, connector, all six conductors. Getting the first one right tells you the gauge and connector choices work before you repeat them nine times.

03 **Run it on the bench**
Level shifter, resistor, cap, one button. Flash the firmware with `WAM_LIGHTS_PIXELS 1` and open the debug `LED WALK` screen — one ring is enough to prove the data path.

04 **Build the power bus first**
Before any daisy-chaining. Confirm 5 V at every tap with nothing else connected.

05 **Add caps to the chain one at a time**
Each addition is a new failure point, and `LED WALK` names the cap it is lighting. A break shows up immediately as everything downstream going dark.

06 **Meter the rail on `LED ALL`**
The only screen that reaches worst-case current. Do this before the table sees players, not after.

07 **Verify every switch on `BUTTONS`**
Press all twelve, the two start buttons included, and confirm each decodes as the right name. This is what catches a swapped pair in the loom.